MINERÍA DE DATOS / INGENIERÍA DE DATOS

Memoria Técnica — Práctica 1: Data Lake con Delta y Limpieza de Datos

Dominio: IBEX 35 & Análisis de Sentimiento Sectorial **Arquitectura:** Data Lake Medallón (PySpark & Delta)

1. Definición del Esquema e Ingesta Bronze (Ej 1-a & Ej 1-b)

Para la ingesta de precios históricos a través de la librería `yfinance`, se ha formalizado una estructura explícita de tipos de datos en PySpark (`StructType`) justificada de la siguiente manera:

Columna	Tipo de Dato	Nulable	Justificación Técnica
`Date`	DateType	No (False)	Clave temporal primaria. Indispensable para cruces y series de tiempo.
`Ticker`	StringType	No (False)	Identificador del activo bursátil (ej. "BBVA.MC"). Clave primaria lógica.
`Close`	DoubleType	No (False)	Variable métrica principal. Representa el valor bursátil final de la sesión.
`Open, High, Low`	DoubleType	Sí (True)	Permiten nulos para prevenir fallos ante sesiones de baja liquidez en el proveedor.
`Volume`	LongType	Sí (True)	Volumen negociado. Se castea de `float` a `Long` usando `coalesce` a 0.
`Dividends, Stock_Splits`	DoubleType	Sí (True)	Eventos corporativos esporádicos. Casi la totalidad de filas registran 0.0 o nulo.

2. Optimización por Column Pruning (Ej 1-c)

Al almacenar los datos en formato Parquet y consultar posteriormente únicamente las columnas `Ticker`, `Date` y `Close`, el motor de Spark aplica *Column Pruning*. Debido a que Parquet organiza la información en *Column Chunks* dentro de *Row Groups* contiguos, Spark únicamente lee los bloques de disco asociados a las tres columnas solicitadas. Esto elimina la necesidad de deserializar y transferir en memoria RAM las columnas secundarias (`Open`, `High`, `Low`, `Volume`, etc.), reduciendo sustancialmente el I/O en disco y el tráfico de red.

3. Persistencia de Tipos en Archivos CSV (Ej 2-a)

Al importar el archivo `sentimiento_ibex.csv` sin inferencia explícita de esquema, todas las columnas ingresan a Spark como `StringType`. Esto ocurre porque el formato CSV carece de metadatos binarios integrados o cabeceras de tipos. Siguiendo los principios de la arquitectura Medallón, la capa Bronze preserva esta información en su estado plano original, postergando el parseo de fechas y decimales a la capa Silver.

4. Estrategia de Integración con Yahoo Finance (Ej 7)

Para cruzar la cotización de los activos con las calificaciones del experto, se extrajo el sector empresarial de cada empresa a través del método `yf.Ticker(ticker).info`. Dado que el análisis de sentimiento se evalúa a nivel sectorial ("Finanzas", "Energía", "Utilities", etc.), se construyó un mapeo de correspondencias hacia la taxonomía estándar internacional en inglés de Yahoo Finance (ej. "Financial Services", "Energy"), habilitando una unión mediante clave compuesta `(Date, Sector)`.

5. Transformación y Limpieza en Capa Silver (Ej 8 & Ej 9)

La construcción de la capa Silver aplica una secuencia de transformaciones higiénicas sobre los datos de sentimiento:

- **Filtrado de anomalías:** Exclusión de cabeceras redundantes e inconsistencias resultantes del append mediante expresiones regulares sobre el campo `Fecha` (`^\d{4}-\d{2}-\d{2}$`).
- **Operación Unpivot:** Conversión del formato ancho original (columnas por sector) a una estructura tabular larga normalizada `(Date, Sector, Sentimiento)`.
- **Casteo seguro con `try_cast`:** Remplazo de comas por puntos en cadenas flotantes y conversión mediante `try_cast(DoubleType())`, asignando `NULL` de forma segura ante entradas no numéricas sin interrumpir el flujo.
- **Deduplicación:** Eliminación de claves duplicadas mediante `dropDuplicates(["Date", "Sector"])`.

6. Transacciones ACID, MERGE y RESTORE en Delta Lake (Ej 3 a Ej 6)

Mediante la carpeta `_delta_log`, Delta Lake aporta características ACID sobre el almacenamiento Parquet:

- **MERGE Idempotente (Ej 4):** La condición de cruce sobre `(Ticker, Date)` combina `whenMatchedUpdateAll()` y `whenNotMatchedInsertAll()` previo descarte de duplicados en origen. La ejecución duplicada valida la idempotencia al no alterar el recuento total de filas (17,754 registros).
- **Schema Enforcement & Evolution (Ej 5-a, 5-b):** La inyección de una columna no declarada (`Close_mas5`) es rechazada por defecto. Su incorporación se habilita mediante la opción `mergeSchema="true"`.
- **Time Travel y RESTORE (Ej 5-c, 5-d, Ej 6):** Tras ejecutar un `DELETE` sobre el ticker `BBVA.MC`, la consulta `versionAsOf` permitió inspeccionar el estado histórico (recuperando las 538 filas originales). Posteriormente, `restoreToVersion()` revirtió físicamente la tabla al commit objetivo preservando la trazabilidad completa en el registro de transacciones.

7. Modelado en Estrella y Exportación a Power BI (Ej 10 & Ej 13)

Se exportan los datos en modelo en estrella particionando la tabla de hechos por 'Ticker'. Esto optimiza la lectura columnar en Power BI al aislar las búsquedas por empresa y reduce sustancialmente el I/O.

El esquema resultante separa la tabla de hechos (`fact_cotizaciones`) de la dimensión descriptiva (`dim_empresa`). El particionado físico por subcarpetas de activo (`partitionBy("Ticker")`) permite a las herramientas de BI aplicar *Partition Pruning*, leyendo únicamente las carpetas del activo consultado.

> **Declaración sobre el Uso de Inteligencia Artificial Generativa**
> Se declara el uso de herramientas de Inteligencia Artificial Generativa (modelos LLM) como soporte técnico para: (1) La depuración de excepciones en el pipeline de PySpark/Delta Lake (resolución del fallo de estrictez en `cast` de Spark 4.0 y prevención de coincidencias múltiples en MERGE), (2) La estructuración del código según el patrón modular MVC (Modelo-Vista-Controlador), y (3) Asistencia en la maquetación formal del presente documento técnico. La totalidad del código, transformaciones, validación de resultados e interpretación de negocio han sido verificadas y evaluadas manualmente.